Steven Hoberg
CSE 431
October 12, 2026

# Insertion vs Merge Sort

## Question 5: Timing Merge sort vs Insertion sort

### 1. Hypothesis

I hypothesize insertion sort will be faster for small inputs (n<10). This is because the overhead associated with splitting the list in merge sort is not worth the guaranteed n*log(n) average case time when the input size is small enough.

### 2. Methods

- System Environment:
    - Language & Version:
        - Python 3.11
        - PyCharm 2023.2.1
    - Hardware:
        - OS: Windows 11
        - CPU: AMD Ryzen 5 2600
        - RAM: 32GB DDR4-3000
- Algorithm Implementation:
    - Standard in-place Insertion Sort from programiz.com
    - Recursive Merge Sort from w3schools
    - Source Code:

```python
import timeit
import random
import matplotlib.pyplot as plt

# https://www.programiz.com/dsa/insertion-sort
def insertion(array):
    for step in range(1, len(array)):
        key = array[step]
        j = step - 1

        # Compare key with each element on the left of it until an element smaller than it is
found
        # For descending order, change key<array[j] to key>array[j].
        while j >= 0 and key < array[j]:
            array[j + 1] = array[j]
            j = j - 1

        # Place key at after the element just smaller than it.
        array[j + 1] = key

# https://www.w3schools.com/python/python_dsa_mergesort.asp
def merge_sort(list):
    def sort(arr):
      if len(arr) <= 1:
        return arr

      mid = len(arr) // 2
      leftHalf = arr[:mid]
      rightHalf = arr[mid:]

      sortedLeft = sort(leftHalf)
      sortedRight = sort(rightHalf)

      return merge(sortedLeft, sortedRight)

    def merge(left, right):
      result = []
      i = j = 0

      while i < len(left) and j < len(right):
        if left[i] < right[j]:
          result.append(left[i])
          i += 1
        else:
          result.append(right[j])
          j += 1

      result.extend(left[i:])
      result.extend(right[j:])

      return result

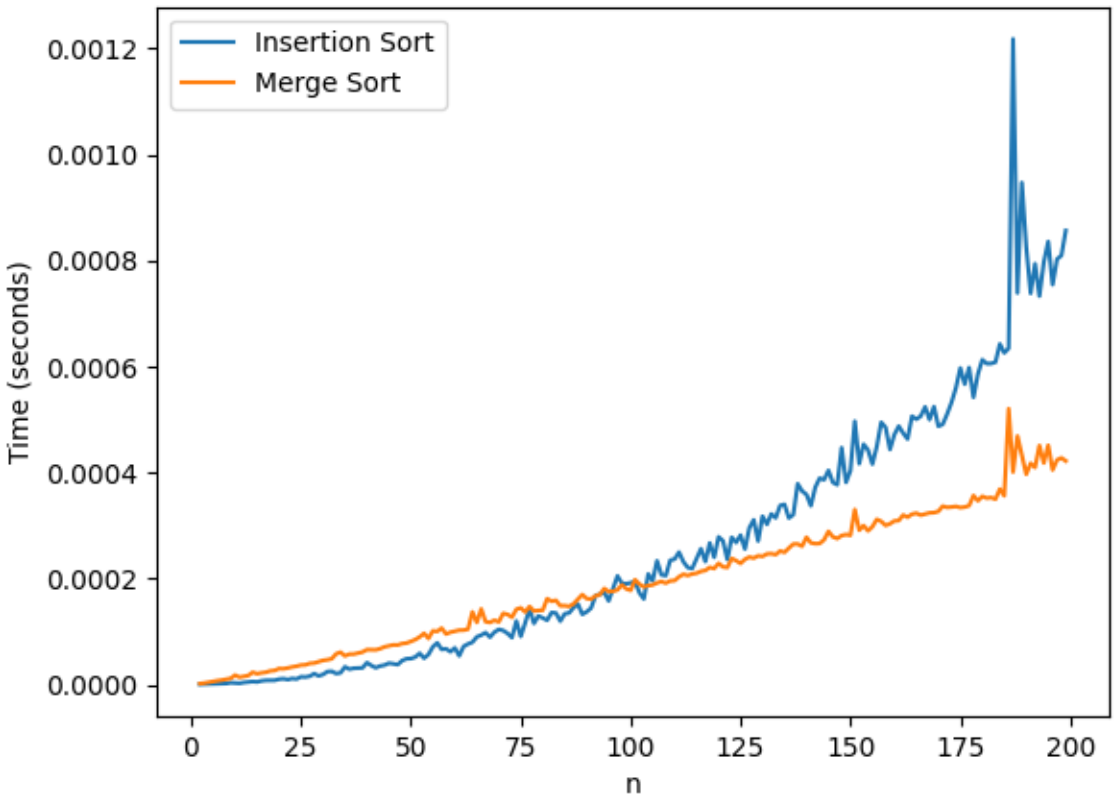
    sorted_list = sort(list)
    return sorted_list
```

```python
if __name__ == "__main__":
    input_max = 200
    trials = 500 # for timeit
    n_values = []
    insertion_times = []
    merge_times = []
    for data_len in range(2, input_max):
        # Fresh unsorted data for this input size
        test_data = [random.randint(0, 100000) for _ in range(data_len)]
        t_ins = timeit.timeit(lambda: insertion(test_data.copy()), number=trials) / trials
        t_mrg = timeit.timeit(lambda: merge_sort(test_data), number=trials) / trials
        n_values.append(data_len)
        insertion_times.append(t_ins)
        merge_times.append(t_mrg)

    # plotting
    plt.plot(n_values, insertion_times, label="Insertion Sort")
    plt.plot(n_values, merge_times, label="Merge Sort")
    plt.xlabel("n")
    plt.ylabel("Time (seconds)")
    plt.legend()
    plt.savefig("sorting_comparison.png")
    plt.show()
```

- Testing Protocol:
    - Ran using basic Python CLI, no flags were used.
    - Input size ranging from 2 <= n <= 200
    - Generated test sets using lists of uniformly distributed random integers (0 to $100{,}000$) for each $n$
    - Used timeit.timeit with $500$ trials per $n$ to minimize OS background noise and small timer inaccuracies.
    - Passed fresh array copies (.copy()) on every iteration to prevent benchmarking pre-sorted data.
    - Recorded average execution time per sort in seconds for each algorithm.

**3. Results**




- Observation:
    - As the input size (n) goes up, sorting takes longer for both implementations, the key difference is how fast.
    - Insertion sort, despite having a worst-case run time of f(n) = O(n^2), is a faster algorithm than merge sort from input sizes 0 to around 100.
    - Insertion sort starts climbing slower but curves quadratically upward while merge sort grows much more steadily. This makes merge sort the faster choice for input sizes n > 100

**4. Discussion**

- Analysis of Crossover:

    The crossover of runtimes happened much later than I initially thought. I hypothesized around n=10 would be the point the overhead of merge sort becomes negligible, but the experiment shows around n=100 is where the two algorithms cross.

This is for two reasons. Insertion sort is in place, meaning it does not need to allocate memory like merge sort does every time it splits the array. Additonally, the recursive structure of merge sort also requires additional state tracking whereas insertion sort is implemented within a single function.

As predicted, the plotted line is not perfectly smooth. There are localized variations, but the repeated time trials minimize this factor. However, it can still be recognized, especially towards higher input sizes. For example, there is a large spike around n = 180.

**5. Conclusions**

Under the tested environmental conditions, insertion sort is faster for $n < 100$, while merge sort is superior for $n > 100$. Between $n = 80$ and $n = 110$, the performance is similar, however the trend of the recorded times points to n = 100 as a reasonable crossover point between performance.

This experiment shows the hidden caveats of relying just on big-O notation to make algorithm decisions. External factors like hardware, language, implementation, and timing can make a large impact on how algorithms perform.

## Question 6: Hybrid Sorting

Below is my implementation for hybrid sorting and then its performance compared to the previous sorting implementations.

Source Code:

```
# https://www.programiz.com/dsa/insertion-sort
def insertion(array):
    for step in range(1, len(array)):
        key = array[step]
        j = step - 1

        # Compare key with each element on the left of it until an element
smaller than it is found
        # For descending order, change key<array[j] to key>array[j].
        while j >= 0 and key < array[j]:
            array[j + 1] = array[j]
            j = j - 1

        # Place key at after the element just smaller than it.
        array[j + 1] = key
    return array

k = 90  # threshold where insertion sort takes over small sub-arrays
```

```python
# https://www.w3schools.com/python/python_dsa_mergesort.asp
def merge_sort(list, use_hybrid = False):

    def sort(arr):
      if len(arr) <= 1:
        return arr

      mid = len(arr) // 2
      leftHalf = arr[:mid]
      rightHalf = arr[mid:]

      sortedLeft = sort(leftHalf)
      sortedRight = sort(rightHalf)

      return merge(sortedLeft, sortedRight)

    def merge(left, right):
      result = []
      i = j = 0

      while i < len(left) and j < len(right):
        if left[i] < right[j]:
          result.append(left[i])
          i += 1
        else:
          result.append(right[j])
          j += 1

      result.extend(left[i:])
      result.extend(right[j:])

      return result

    def hybrid_sort(arr):
        if len(arr) <= k:
            return insertion(arr)
        mid = len(arr) // 2
        return merge(hybrid_sort(arr[:mid]), hybrid_sort(arr[mid:]))

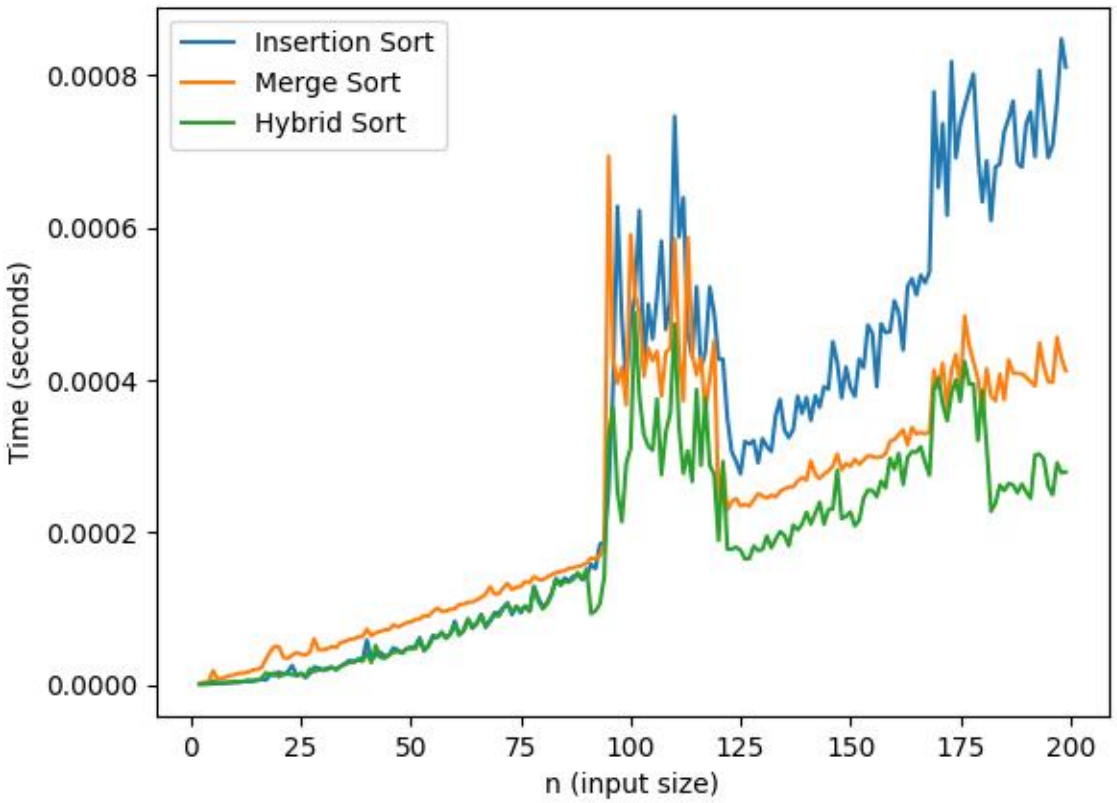
    if use_hybrid:
        return hybrid_sort(list)
    else:
        return sort(list)




k = 90 # constant for hybrid crossover list length
if __name__ == "__main__":
    input_max = 200
    trials = 500 # for timeit
    n_values = []
    insertion_times = []
    merge_times = []
    hybrid_times = []
```

```
    for data_len in range(2, input_max):
        # Fresh unsorted data for this input size
        test_data = [random.randint(0, 100000) for _ in range(data_len)]
        t_ins = timeit.timeit(lambda: insertion(test_data.copy()),
number=trials) / trials
        t_mrg = timeit.timeit(lambda: merge_sort(test_data.copy()),
number=trials) / trials
        t_hyb = timeit.timeit(lambda: merge_sort(test_data.copy(),
use_hybrid=True), number=trials) / trials
        n_values.append(data_len)
        insertion_times.append(t_ins)
        merge_times.append(t_mrg)
        hybrid_times.append(t_hyb)

    # plotting
    plt.plot(n_values, insertion_times, label="Insertion Sort")
    plt.plot(n_values, merge_times, label="Merge Sort")
    plt.plot(n_values, hybrid_times, label="Hybrid Sort")
    plt.xlabel("n (input size)")
    plt.ylabel("Time (seconds)")
    plt.legend()
    plt.savefig("sorting_comparison.png")
    plt.show()
```



As you can see by the green line, it is consistently equal to or faster than the non-hybrid approaches. The placement of k seems to stay the same, this is expected since the underlying sorting methods and overhead between them did not change. However, the runtime for all implementations shoots up around k, this is likely an allocation fluke.